Object-Oriented Programming & Modules

Liting Zhang

University of West Florida

Overview

- Objects, classes, encapsulation
- Inheritance and composition
- Dynamic dispatch and method lookup
- Interfaces vs abstract classes
- Mixins and traits (conceptual)
- Modules, packages, visibility, namespaces
- Industry angle: service and library design, package structure, code organization
- Languages: Java, TypeScript classes and modules, Python modules and packages

Learning goals

After this lecture, you should be able to

- Explain objects, classes, encapsulation, inheritance, and composition
- Predict behavior under dynamic dispatch and explain method lookup
- Choose between interfaces and abstract classes for a design goal
- Describe mixins and traits at a conceptual level
- Explain modules and packages, including visibility and namespaces
- Organize code using common industry package structures and layering

Core idea: structure code around responsibilities

OOP and modules are about managing complexity:

- Encapsulation: hide representation details behind methods
- Stable interfaces: depend on contracts, not implementations
- Separation of concerns: split features into modules/packages
- Evolution: add features without rewriting everything

A useful mental model:

- Classes define capabilities and state
- Interfaces define contracts
- Modules/packages define boundaries and visibility

Objects and Classes

Objects and classes

Object

- A runtime entity that bundles state and behavior
- State stored in fields, behavior provided by methods
- Multiple objects can share the same class

Class

- A blueprint for creating objects
- Defines fields and methods
- Often also defines invariants that should always hold

Encapsulation connects these: interact through methods, not raw fields.

Java: class and object basics

```
1  class BankAccount {
2    private int balanceCents; //Encapsulation: balanceCents is private;
3    BankAccount(int initialCents) {
4      if (initialCents < 0) throw new IllegalArgumentException();
5      balanceCents = initialCents;
6    }
7    void deposit(int cents) {
8      if (cents <= 0) throw new IllegalArgumentException();
9      balanceCents += cents;
10   }
11   boolean withdraw(int cents) {
12     if (cents <= 0) throw new IllegalArgumentException();
13     if (cents > balanceCents) return false;
14     balanceCents -= cents;
15     return true;
16   }
17   int getBalanceCents() {
18     return balanceCents;
19   }
20 }
21 class Demo {
22   public static void main(String[] args) {
23     BankAccount a = new BankAccount(1000);
24     a.deposit(250);
25     System.out.println(a.getBalanceCents());
26   }
```

TypeScript: class basics

TypeScript compiles to JavaScript classes; access modifiers are checked by TypeScript.

```
1  class BankAccount {
2    private balanceCents: number;
3    constructor(initialCents: number) {
4      if (initialCents < 0) throw new Error("bad init");
5      this.balanceCents = initialCents;
6    }
7    deposit(cents: number): void {
8      if (cents <= 0) throw new Error("bad deposit");
9      this.balanceCents += cents;
10   }
11   getBalanceCents(): number {
12     return this.balanceCents;
13   }
14 }
15 const a = new BankAccount(1000);
16 a.deposit(250);
17 console.log(a.getBalanceCents());
```

Python: class basics

Python uses classes and objects with a different visibility culture.

```
1  class BankAccount:
2    def __init__(self, initial_cents: int):
3      if initial_cents < 0:
4        raise ValueError("bad init")
5      self._balance_cents = initial_cents  # convention: internal
6    def deposit(self, cents: int) -> None:
7      if cents <= 0:
8        raise ValueError("bad deposit")
9      self._balance_cents += cents
10   def balance_cents(self) -> int:
11     return self._balance_cent
12 a = BankAccount(1000)
13 a.deposit(250)
14 print(a.balance_cents())
```

Underscore naming signals intended privacy; it is a convention, not enforcement.

Encapsulation: representation and invariants

Encapsulation is not only about hiding fields:

- Protect invariants by controlling updates
- Reduce coupling by exposing a stable interface
- Allow internal representation to change without breaking callers

Common invariants:

- Balance never negative
- Collection contains no duplicates
- State transitions follow allowed rules

Practice 1: fix an encapsulation leak (Java)

Identify what breaks the invariant and propose a fix.

```java
class Account {
  public int balanceCents;  // intended invariant: non-negative
  public Account(int initial) {
    balanceCents = initial; }
  public void deposit(int cents) {
    balanceCents += cents; }
}
class Demo {
  public static void main(String[] args) {
    Account a = new Account(100);
    a.balanceCents = -999;
    System.out.println(a.balanceCents);
  }
}
```

- What went wrong
- What change enforces the invariant

Practice 1 Answer: fix an encapsulation leak (Java)

What went wrong

- balanceCents is public, so callers can assign any value.
- The class cannot enforce the non-negative invariant.

One correct fix: make state private and validate updates

```java
class Account {
  private int balanceCents;
  public Account(int initial) {
    if (initial < 0) throw new IllegalArgumentException();
    balanceCents = initial;
  }
  public void deposit(int cents) {
    if (cents <= 0) throw new IllegalArgumentException();
    balanceCents += cents;
  }
  public boolean withdraw(int cents) {
    if (cents <= 0) throw new IllegalArgumentException();
    if (cents > balanceCents) return false;
    balanceCents -= cents;
    return true;
  }
  public int getBalanceCents() { return balanceCents; }
}
```

Result: callers cannot write the field directly; invariants live in methods.

Inheritance and Composition

Inheritance

Inheritance models an is-a relationship:

- A subclass reuses and extends a superclass interface/implementation
- Shared behavior can be placed in a base class
- Dynamic dispatch chooses overridden methods at runtime

Risks:

- Tight coupling to base class internals
- Fragile base class problem when base changes
- Inheritance can encode the wrong relationship (has-a mistaken as is-a)

Composition

Composition models a has-a relationship:

- Build objects from other objects
- Delegate work to a contained component
- Often easier to test and evolve than deep inheritance hierarchies

A common guideline:

- Prefer composition for sharing behavior across unrelated types
- Use inheritance when there is a stable is-a relationship and polymorphism is central

Java: inheritance example

```
1  class Shape {
2    double area() {
3      return 0.0; }
4  }
5  class Circle extends Shape {
6    private double r;
7    Circle(double r) {
8      this.r = r; }
9    @Override double area() {
10     return Math.PI * r * r; }
11 }
12 class Rect extends Shape {
13   private double w, h;
14   Rect(double w, double h) {
15     this.w = w; this.h = h; }
16   @Override double area() {
17     return w * h; }
18 }
```

Shape is a common supertype, enabling polymorphic code over Shape.

TypeScript: composition by delegation

Instead of inheriting, delegate to a component object.

```
type Logger = { log: (msg: string) => void };
class Service {
  private logger: Logger;
  constructor(logger: Logger) {
    this.logger = logger;
  }
  handle(requestId: string): void {
    this.logger.log("handling " + requestId);
  }
}
const consoleLogger: Logger = {
  log: (m) => console.log(m) };
const s = new Service(consoleLogger);
s.handle("r1");
```

Service has-a logger and delegates logging behavior.

Practice 2: inheritance vs composition choice

You want to add caching to a data source.
Option A: inherit from DataSource and override read.
Option B: store a DataSource inside a CacheDataSource and delegate reads.
Task:

- Which option better supports adding caching to many different data sources
- What is the risk of deep inheritance for this feature

Practice 2 Answer: inheritance vs composition choice

A reasonable answer:

- Composition is typically better for caching because caching is an add-on feature that can wrap many unrelated data sources.
- With composition, CacheDataSource can wrap any DataSource instance and delegate to it.
- Deep inheritance risks coupling caching logic to one base class design and can explode into many subclasses:
  - CachedFileDataSource, CachedDbDataSource, CachedApiDataSource, plus variants
- Composition supports stacking features like caching + retry + logging using wrappers.

Dynamic Dispatch and Method Lookup

Dynamic dispatch

Dynamic dispatch means the method implementation is chosen at runtime based on the receiver object's class.

- Call site decides which method name to invoke
- Runtime decides which override to run
- Enables polymorphism: same call works for many concrete types

This is central to typical OOP designs: code depends on a base type or interface, not a concrete class.

Java: dynamic dispatch example

```java
class Animal {
  String speak() { return "???"; }
}
class Dog extends Animal {
  @Override String speak() { return "woof"; }
}
class Cat extends Animal {
  @Override String speak() { return "meow"; }
}
class Demo {
  static void talk(Animal a) {
    System.out.println(a.speak());
  }
  public static void main(String[] args) {
    Animal a1 = new Dog();
    Animal a2 = new Cat();
    talk(a1);
    talk(a2);
  }
}
```

Practice 3: predict dispatch output (Java)

Predict the output.

```java
class A {
  String f() { return "A.f"; }
}
class B extends A {
  @Override String f() { return "B.f"; }
  String g() { return "B.g"; }
}
class Demo {
  static void run(A x) {
    System.out.println(x.f());
  }
  public static void main(String[] args) {
    A x = new B();
    run(x);
  }
}
```

- Which method body runs and why

Practice 3 Answer: predict dispatch output (Java)

Output

- B.f

Explanation

- The static type of x in run is A, so the call is x.f().
- The dynamic type of the object is B, so dynamic dispatch selects B.f at runtime.
- Method g is not relevant because run never calls g.

Method lookup intuition

A practical way to explain method selection:

- The compiler checks whether the method name exists on the static type
- The runtime selects the most specific override on the actual object

This separates:

- Type checking time: is the call allowed
- Runtime dispatch: which body executes

Interfaces vs Abstract Classes

Interfaces

Interface

- A contract: method signatures that implementers must provide
- Supports polymorphism without inheriting implementation
- Encourages programming to an interface

Typical uses:

- Plug-in points and dependency injection
- Multiple implementations behind a stable API
- Test doubles: fake implementations for testing

Abstract classes

Abstract class

- A partial implementation plus an abstract contract
- Can define shared fields, constructors, and non-abstract methods
- Often used for a family of closely related classes with shared state or behavior

Typical tradeoff:

- Abstract class provides reuse
- Interface provides decoupling and flexibility

Java: interface vs abstract class

Interface example:

```
interface PaymentProcessor {
  boolean charge(int cents);
}
class StripeProcessor implements PaymentProcessor {
  public boolean charge(int cents) { return cents > 0; }
}
```

Abstract class example:

```
abstract class BaseProcessor {
  protected int attempts = 0;
  public boolean chargeWithRetry(int cents) {
    attempts++;
    return charge(cents);
  }
  protected abstract boolean charge(int cents);
}
```

TypeScript: interfaces and classes

TypeScript uses structural typing: if it has the right shape, it matches the interface.

```
1  interface PaymentProcessor {
2    charge(cents: number): boolean;
3  }
4
5  class StripeProcessor implements PaymentProcessor {
6    charge(cents: number): boolean {
7      return cents > 0;
8    }
9  }
10
11 function checkout(p: PaymentProcessor): void {
12   console.log(p.charge(250));
13 }
14 checkout(new StripeProcessor());
```

Practice 4: choose interface or abstract class

You are designing a library with multiple implementations.
Scenario A: you want many unrelated implementations and minimal coupling.
Scenario B: you want to share state, constructors, and helper methods among close variants.
Task:

- For A, choose interface or abstract class and justify
- For B, choose interface or abstract class and justify

Practice 4 Answer: choose interface or abstract class

A reasonable answer:

- Scenario A: interface
  - Many unrelated implementations benefit from a pure contract
  - Callers depend on the interface, not shared base state
- Scenario B: abstract class
  - Shared state and constructors belong in a base type
  - Helper methods reduce duplication among closely related variants

Practical guideline:

- Default to interface for public extension points
- Use abstract classes inside a library when shared implementation is central

Mixins and Traits

Mixins and traits (conceptual)

Mixin

- A reusable bundle of methods that can be combined into many classes
- Often used to share behavior without forming a deep inheritance chain

Trait

- Similar goal: compose behavior from pieces
- Often includes conflict rules for method name collisions

Many languages implement some form of this idea:

- TypeScript uses mixin patterns with functions that extend classes
- Python supports multiple inheritance, which can be used mixin-style
- Java uses interfaces with default methods as a limited form of reusable behavior

TypeScript: simple mixin pattern

A mixin can add methods to a base class.

```
type Ctor<T> = new (...args: any[]) => T; //constructor type

function Timestamped<TBase extends Ctor<{}>>(Base: TBase) {
  return class extends Base {
    createdAt = new Date();
    ageMs(): number {
      return Date.now() - this.createdAt.getTime(); }
  };
}

class User {
  constructor(public name: string) {}
}
const TimestampedUser = Timestamped(User);
const u = new TimestampedUser("Ada");
console.log(u.name);
console.log(u.ageMs());
```

Python: mixin-style multiple inheritance

Python can compose behavior using mixin classes.

```
class JsonMixin:
    def to_json(self) -> str:
        import json
        return json.dumps(self.__dict__)

class User(JsonMixin):
    def __init__(self, name: str):
        self.name = name

u = User("Ada")
print(u.to_json())
```

Multiple inheritance should be used carefully to avoid complex method resolution.

Practice 5: mixin tradeoffs

Consider using mixins to add logging and caching behaviors.

- What problem do mixins solve compared to inheritance
- What new problem can mixins introduce
- Name one safer alternative for combining features

Practice 5 Answer: mixin tradeoffs

A reasonable answer:

- Mixins solve code reuse across many unrelated classes without deep inheritance.
- Mixins can introduce name conflicts and hard-to-understand method resolution.
- A safer alternative is composition using wrappers/delegation:
  - a service has-a logger
  - a data source has-a cache

Modules and Packages

Modules and namespaces

Module

- A unit of code organization: a file or compilation unit
- Defines what is exported and what stays internal
- Prevents name collisions by using imports rather than global names

Package

- A collection of related modules under a namespace
- Defines visibility and dependency boundaries

A common goal: keep internal helpers private and expose a small public API surface.

Visibility and access control across languages

Java

- public, protected, package-private (default), private
- package-private is a strong tool for internal APIs

TypeScript

- public, protected, private checked by compiler
- runtime is JavaScript; enforcement is mostly at compile time

Python

- underscore conventions: _internal, __name mangling
- relies on discipline and documentation rather than strict enforcement

TypeScript modules: export and import

A typical two-file module design.

File math.ts:

```ts
export function add(a: number, b: number): number {
  return a + b;
}
function helper(x: number): number {
  return x * 2;
}
export function addThenDouble(a: number, b: number): number {
  return helper(add(a, b));
}
```

File main.ts:

```ts
import { addThenDouble } from "./math";

console.log(addThenDouble(2, 3));
```

helper is not exported, so it is internal to the module.

Practice 6: TypeScript import name conflicts

Suppose you import two modules with the same exported name.

```
1 import { parse } from "./json";
2 import { parse } from "./xml"; // conflict
```

Task:

- Fix the imports so both functions are usable
- Show a call to each parse function

Practice 6 Answer: TypeScript import name conflicts

Use import aliasing:

```
1 import { parse as parseJson } from "./json";
2 import { parse as parseXml } from "./xml";
3
4 const a = parseJson("{\"x\":1}");
5 const b = parseXml("<x>1</x>");
6
7 console.log(a, b);
```

Aliasing resolves conflicts while keeping call sites readable.

Java packages: organization and visibility

Java uses packages as namespaces and visibility boundaries.
File src/com/example/app/User.java:

```java
package com.example.app;

class UserRepo { // package-private: internal to com.example.app
  User find(String id) { return new User(id); }
}

public class User {
  public final String id;
  public User(String id) { this.id = id; }
}
```

Package-private classes and methods support internal APIs without exposing them publicly.

Python modules and packages

Python module is a .py file; a package is a directory.

Example structure:

```
app/
  __init__.py
  service.py
  util.py
```

Example imports:

```
from app.service import run
from app import util
```

__init__.py marks a directory as a package and can also define the package public API.

Practice 7: Python module entry points

You have:

```
app/
  __init__.py
  main.py
  util.py
```

main.py wants to call util.double.

Task:

- Write the import statement inside main.py
- Explain one common mistake that causes import errors when running as a script

Practice 7 Answer: Python module entry points

A correct import inside app/main.py:

```
1 from app.util import double
```

Common mistake:

- Running app/main.py directly can change module resolution because app may not be treated as a package.
- A common robust approach is running as a module:

```
1 python -m app.main
```

Industry Code Organization

Typical layering in services and libraries

A common service structure:

- API layer: controllers/handlers, request parsing
- Domain layer: core business rules, pure logic where possible
- Data layer: repositories, persistence, external APIs
- Infrastructure: logging, metrics, configuration

Goal: dependencies point inward.

- Domain should not depend on database libraries directly
- Use interfaces to invert dependencies for testability

Package structure strategies

Two common approaches:

Package by layer

- controllers/, services/, repositories/, models/
- Easy to find technical concerns, can scatter feature code

Package by feature

- users/, orders/, billing/ each contains controller/service/repo
- Easier to work on a feature end-to-end, can duplicate infrastructure patterns

Both can work; consistency matters more than the choice.

Practice 8: package boundary design

You have a library that exposes a public API and also contains internal helpers.
Task:

- Name one technique in Java to keep helpers internal
- Name one technique in TypeScript to keep helpers internal
- Name one technique in Python to signal internal helpers

Practice 8 Answer: package boundary design

A reasonable answer:

- Java: use package-private classes/methods and expose only public types in the package API.
- TypeScript: do not export internal helpers from a module, export only intended public functions/types.
- Python: use underscore naming for internal functions and place them in non-exported modules; document the public API.

Review

Summary: what to remember

- Encapsulation protects invariants and reduces coupling
- Inheritance supports is-a polymorphism but increases coupling
- Composition supports has-a reuse and often scales better for feature combinations
- Dynamic dispatch selects method bodies at runtime based on the receiver object
- Interfaces define contracts; abstract classes combine contracts with shared implementation
- Mixins and traits aim to compose behavior, but can complicate method resolution
- Modules and packages create namespaces and boundaries for visibility and organization
- Industry codebases rely on clear APIs, stable package boundaries, and consistent structure

Review questions

- What is the difference between exposing fields and exposing methods for the same state
- When does composition beat inheritance for extending behavior
- In dynamic dispatch, what is checked at compile time and what is decided at runtime
- When would you prefer an interface to an abstract class in a library API
- What does a module export boundary do for long-term maintainability
- Which package structure fits your next project and why

Review question answers

What is the difference between exposing fields and exposing methods for the same state

- Exposing fields lets callers read and write state directly, which can break invariants and tightly couples callers to representation.
- Exposing methods lets the class validate updates, preserve invariants, and change internal representation without changing callers.

When does composition beat inheritance for extending behavior

- When the added behavior is an optional feature that should apply to many unrelated types (logging, caching, retries, metrics).
- When you want to combine multiple features without creating many subclasses.
- When you want to avoid coupling to a base class implementation and keep unit testing simpler.

Review question answers

In dynamic dispatch, what is checked at compile time and what is decided at runtime

- Compile time: the method name must exist on the static type, and the argument/return types must match the method signature.
- Runtime: which overriding method body runs is chosen based on the receiver object's actual class.

When would you prefer an interface to an abstract class in a library API

- When you want many independent implementations with minimal coupling to shared code.
- When you want callers to depend only on a contract and not inherit state/constructors.
- When you want easier testing and mocking via alternate implementations.

Review question answers

What does a module export boundary do for long-term maintainability

- It defines a small public API surface that other code depends on.
- It keeps helpers internal so they can be changed without breaking external users.
- It reduces name collisions and makes dependencies explicit through imports.

Which package structure fits your next project and why

- Package by feature fits when teams work end-to-end on features and want related code in one place.
- Package by layer fits when a project emphasizes technical layering (controllers/services/repos) and consistent cross-cutting patterns.
- A common compromise is feature packages with shared infrastructure modules for logging/config/utilities.